

# Installation

To install the CRAN release version of `ctmle`:

```
install.packages('ctmle')
```

To install the development version (requires the devtools package):

```
devtools::install_github('jucheng1992/ctmle')
```

## Collaborative Targeted Maximum Likelihood Estimation

In this package, we implemented the general template of C-TMLE, for estimation of average additive treatment effect (ATE). The package also offers the functions for discrete C-TMLE, which could be used for variable selection, and C-TMLE for model selection of LASSO.

### C-TMLE for variable selection

In this section, we start with examples of discrete C-TMLE for variable selection, using greedy forward searching, and scalable discrete C-TMLE with pre-ordering option.

```
library(ctmle)
```

```
## Loading required package: SuperLearner
```

```
## Loading required package: nnls
```

```
## Super Learner
```

```
## version: 2.0-25
```

```
## Package created on 2019-08-05
```

```
## Loading required package: tmle
```

```
## Loading required package: glmnet
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 3.0-1
```

```
## welcome to the tmle package, version 1.4.0.1  
##  
## Use tmleNews() to see details on changes and bug fixes
```

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':  
##  
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```

set.seed(123)

N <- 1000
p = 5
Wmat <- matrix(rnorm(N * p), ncol = p)
beta1 <- 4+2*Wmat[,1]+2*Wmat[,2]+2*Wmat[,5]
beta0 <- 2+2*Wmat[,1]+2*Wmat[,2]+2*Wmat[,5]
tau <- 2
gcoef <- matrix(c(-1,-1,rep(-(3/((p)-2)),(p)-2)),ncol=1)
W <- as.matrix(Wmat)

g <- 1/(1+exp(W%*%gcoef /3))
A <- rbinom(N, 1, prob = g)

epsilon <-rnorm(N, 0, 1)
Y <- beta0 + tau * A + epsilon

# With initial estimate of Q
Q <- cbind(rep(mean(Y[A == 0]), N), rep(mean(Y[A == 1]), N))

time_greedy <- system.time(
  ctmle_discrete_fit1 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat),
  preOrder = FALSE, detailed = TRUE)
)
ctmle_discrete_fit2 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat),
  preOrder = FALSE, detailed = TRUE)

time_preorder <- system.time(
  ctmle_discrete_fit3 <- ctmleDiscrete(Y = Y, A = A, W = data.frame(Wmat),
  preOrder = TRUE,
  order = rev(1:p), detailed = TRUE)
)

```

Scalable (discrete) C-TMLE takes much less computation time:

```
time_greedy
```

```
##      user  system elapsed
##    1.149   0.067   1.220
```

```
time_preorder
```

```
##      user  system elapsed  
##    0.754    0.017    0.772
```

Show the brief results from greedy CTMLE:

```
ctmle_discrete_fit1
```

```
## C-TMLE result:  
## parameter estimate: 1.99642  
## estimated variance: 0.00905  
##                p-value: <2e-16  
## 95% conf interval: (1.80998, 2.18287)
```

Summary function offers detail information of which variable is selected.

```
summary(ctmle_discrete_fit1)
```

```
##
## Number of candidate TMLE estimators created: 6
## A candidate TMLE estimator was created at each move, as each new term
## was incorporated into the model for g.
## -----
##      term added cleverCovar estimate cv-RSS cv-varIC cv-penRSS
## cand 1 (intercept)      1      4.22   19.8   0.0794   13991
## cand 2           x2      1      3.22   19.5   0.0855   13751
## cand 3           x5      1      2.61   19.0   0.0897   13414
## cand 4           x1      1      2.00   18.2   0.0956   12877
## cand 5           x4      2      1.99   18.3   0.1112   12930
## cand 6           x3      3      2.01   18.3   0.1018   12930
## -----
## Selected TMLE estimator is candidate 4
##
## Each TMLE candidate was created by fluctuating the initial fit,  $Q_0(A,W)=$ 
##
## cand 1:  $Q_1(A,W) = Q_0(A,W) + \text{epsilon}_{1a} * h_{1a}$ 
##            $h_{1a}$  is based on an intercept-only model for treatment mecha
##
## cand 2:  $Q_2(A,W) = Q_0(A,W) + \text{epsilon}_{1b} * h_{1b}$ 
##            $h_{1b}$  is based on a treatment mechanism model containing covar
##
## cand 3:  $Q_3(A,W) = Q_0(A,W) + \text{epsilon}_{1c} * h_{1c}$ 
##            $h_{1c}$  is based on a treatment mechanism model containing covar
##
## cand 4:  $Q_4(A,W) = Q_0(A,W) + \text{epsilon}_{1d} * h_{1d}$ 
##            $h_{1d}$  is based on a treatment mechanism model containing covar
##
## cand 5:  $Q_5(A,W) = Q_0(A,W) + \text{epsilon}_{1d} * h_{1d} + \text{epsilon}_2 * h_2$ 
##            $h_2$  is based on a treatment mechanism model containing covar
##
## cand 6:  $Q_6(A,W) = Q_0(A,W) + \text{epsilon}_{1d} * h_{1d} + \text{epsilon}_2 * h_2 + \text{epsilon}_3 * h_3$ 
##            $h_3$  is based on a treatment mechanism model containing covar
##
## -----
## C-TMLE result:
## parameter estimate: 1.99642
## estimated variance: 0.00905
## p-value: <2e-16
## 95% conf interval: (1.80998, 2.18287)
```

# C-TMLE LASSO for model selection of LASSO

In this section, we introduce the C-TMLE algorithms for model selection of LASSO in the estimation of propensity core, and for simplicity we call them LASSO C-TMLE algorithm. We have three variations of C-TMLE LASSO algorithms, see technical details in the corresponding references.

```
# Generate high-dimensional data
set.seed(123)

N <- 1000
p = 100
wmat <- matrix(rnorm(N * p), ncol = p)
beta1 <- 4+2*wmat[,1]+2*wmat[,2]+2*wmat[,5]+2*wmat[,6]+2*wmat[,8]
beta0 <- 2+2*wmat[,1]+2*wmat[,2]+2*wmat[,5]+2*wmat[,6]+2*wmat[,8]
tau <- 2
gcoef <- matrix(c(-1,-1,rep(-(3/((p)-2)), (p)-2)), ncol=1)
W <- as.matrix(wmat)

g <- 1/(1+exp(W%*%gcoef /3))
A <- rbinom(N, 1, prob = g)

epsilon <- rnorm(N, 0, 1)
Y <- beta0 + tau * A + epsilon

# With initial estimate of Q
Q <- cbind(rep(mean(Y[A == 0]), N), rep(mean(Y[A == 1]), N))

glmnet_fit <- cv.glmnet(y = A, x = W, family = 'binomial', nlambda = 20)
```

We suggest start build a sequence of lambdas from the lambda selected by cross-validation, as the model selected by cv.glmnet would over-smooth w.r.t. the target parameter.

```
lambdas <- glmnet_fit$lambda[(which(glmnet_fit$lambda==glmnet_fit$lambda.min
```

We fit C-TMLE1 algorithm by feed the algorithm with a vector of lambda, in decreasing order:

```
time_ctmlelasso1 <- system.time(
  ctmle_fit1 <- ctmleGlmnet(Y = Y, A = A,
                           W = data.frame(W = W),
                           Q = Q, lambdas = lambdas, ctmletype=1,
                           family="gaussian", gbound=0.025, v=5)
)
```

## We fit C-TMLE2 algorithm

```
time_ctmlelasso2 <- system.time(
  ctmle_fit2 <- ctmleGlmnet(Y = Y, A = A,
                           W = data.frame(W = W),
                           Q = Q, lambdas = lambdas, ctmletype=2,
                           family="gaussian",gbound=0.025, v=5)
)
```

For C-TMLE3, we need two gn estimators, one with lambda selected by cross-validation, and the other with lambda slightly different from the selected lambda:

```
gcv <- stats::predict(glmnet_fit, newx=W, s="lambda.min",type="response")
gcv <- bound(gcv,c(0.025,0.975))

s_prev <- glmnet_fit$lambda[(which(glmnet_fit$lambda == glmnet_fit$lambda.r
gcvPrev <- stats::predict(glmnet_fit,newx = W,s = s_prev,type="response")
gcvPrev <- bound(gcvPrev,c(0.025,0.975))

time_ctmlelasso3 <- system.time(
  ctmle_fit3 <- ctmleGlmnet(Y = Y, A = A, W = W, Q = Q,
                           ctmletype=3, glw = gcv, glwPrev = gcvPrev,
                           family="gaussian",
                           gbound=0.025, v = 5)
)
```

```
## Warning in c(-1.96, 1.96) * sqrt(var.psi): Recycling array of length 1 i
## Use c() or as.vector() instead.
```

Let's compare the running time for each LASSO-C-TMLE

```
time_ctmlelasso1
```

```
##      user  system elapsed
## 14.745    0.296   15.084
```

```
time_ctmlelasso2
```

```
##      user  system elapsed
## 19.292    0.453   19.829
```

```
time_ctmlelasso3
```

```
##      user  system elapsed  
##    0.005    0.000    0.005
```

Finally, we compared three C-TMLE estimates:

```
ctmle_fit1
```

```
## C-TMLE result:  
## parameter estimate: 2.19644  
## estimated variance: 0.10065  
##           p-value: 4.4125e-12  
## 95% conf interval: (1.57462, 2.81826)
```

```
ctmle_fit2
```

```
## C-TMLE result:  
## parameter estimate: 2.16669  
## estimated variance: 0.05327  
##           p-value: <2e-16  
## 95% conf interval: (1.71429, 2.61908)
```

```
ctmle_fit3
```

```
## C-TMLE result:  
## parameter estimate: 2.02388  
## estimated variance: 0.04972  
##           p-value: <2e-16  
## 95% conf interval: (1.58684, 2.46093)
```

Show which regularization parameter (lambda) is selected by C-TMLE1:

```
lambdas[ctmle_fit1$best_k]
```

```
## [1] 0.00271545
```

In comparison, show which regularization parameter (lambda) is selected by cv.glmnet:

```
glmnet_fit$lambda.min
```



```
## [1] 0.03065303
```

## Advanced topic: the general template of C-TMLE

In this section, we briefly introduce the general template of C-TMLE. In this function, the `gn` candidates could be a user-specified matrix, each column stand for the estimated PS for each unit. The estimators should be ordered by their empirical fit.

As C-TMLE requires cross-validation, it needs two `gn` estimate: one from cross-validated prediction, one from a vanilla prediction. For example, consider 5-folds cross-validation, where argument `folds` is the list of indices for each folds, then the  $(i,j)$ -th element in input `gn_candidates_cv` should be the predicted value of  $i$ -th unit, predicted by  $j$ -th unit, trained by other 4 folds where all of them do not contain  $i$ -th unit. `gn_candidates` should be just the predicted PS for each estimator trained on the whole data.

We could easily use `SuperLearner` package and `build_gn_seq` function to easily achieve this:

```

lasso_fit <- cv.glmnet(x = as.matrix(W), y = A, alpha = 1, nlambda = 100, r
lasso_lambdas <- lasso_fit$lambda[lasso_fit$lambda <= lasso_fit$lambda.min]

# Build SL template for glmnet
SL.glmnet_new <- function(Y, X, newX, family, obsweights, id, alpha = 1,
                          nlambda = 100, lambda = 0,...){
  # browser()
  if (!is.matrix(X)) {
    X <- model.matrix(~-1 + ., X)
    newX <- model.matrix(~-1 + ., newX)
  }
  fit <- glmnet::glmnet(x = X, y = Y,
                        lambda = lambda,
                        family = family$family, alpha = alpha)
  pred <- predict(fit, newX = newX, type = "response")
  fit <- list(object = fit)
  class(fit) <- "SL.glmnet"
  out <- list(pred = pred, fit = fit)
  return(out)
}

# Use a sequence of estimator to build gn sequence:
SL.cv1lasso <- function (... , alpha = 1, lambda = lasso_lambdas[1]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv2lasso <- function (... , alpha = 1, lambda = lasso_lambdas[2]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv3lasso <- function (... , alpha = 1, lambda = lasso_lambdas[3]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.cv4lasso <- function (... , alpha = 1, lambda = lasso_lambdas[4]){
  SL.glmnet_new(... , alpha = alpha, lambda = lambda)
}

SL.library = c('SL.cv1lasso', 'SL.cv2lasso', 'SL.cv3lasso', 'SL.cv4lasso',

```

Construct the object `fold`s, which is a list of indices for each fold

```

V = 5
folds <-by(sample(1:N,N), rep(1:V, length=N), list)

```

Use `fold`s and SuperLearner template to compute `gn_candidates` and `gn_candidates_cv`

```
gn_seq <- build_gn_seq(A = A, w = w, SL.library = SL.library, folds = folds
```



```
## Number of covariates in A11 is: 100
```

```
## CV SL.cv1lasso_A11
```

```
## CV SL.cv2lasso_A11
```

```
## CV SL.cv3lasso_A11
```

```
## CV SL.cv4lasso_A11
```

```
## CV SL.glm_A11
```

```
## Number of covariates in A11 is: 100
```

```
## CV SL.cv1lasso_A11
```

```
## CV SL.cv2lasso_A11
```

```
## CV SL.cv3lasso_A11
```

```
## CV SL.cv4lasso_A11
```

```
## CV SL.glm_A11
```

```
## Number of covariates in A11 is: 100
```

```
## CV SL.cv1lasso_A11
```

```
## CV SL.cv2lasso_A11
```

```
## CV SL.cv3lasso_A11
```

```
## CV SL.cv4lasso_A11
```

```
## CV SL.glm_A11
```

```
## Number of covariates in A11 is: 100
```

```
## CV SL.cv1lasso_A11
```

```
## CV SL.cv2lasso_A11
```

```
## CV SL.cv3lasso_A11
```

```
## CV SL.cv4lasso_A11
```

```
## CV SL.glm_A11
```

```
## Number of covariates in A11 is: 100
```

```
## CV SL.cv1lasso_A11
```

```
## CV SL.cv2lasso_A11
```

```
## CV SL.cv3lasso_A11
```

```
## CV SL.cv4lasso_A11
```

```
## CV SL.glm_A11
```

```
## Non-Negative least squares convergence: TRUE
```

```
## full SL.cv1lasso_A11
```

```
## full SL.cv2lasso_A11
```

```
## full SL.cv3lasso_A11
```

```
## full SL.cv4lasso_All
```

```
## full SL.glm_All
```

Lets look at the output of `build_gn_seq`

```
gn_seq$gn_candidates %>% dim
```

```
## [1] 1000    5
```

```
gn_seq$gn_candidates_cv %>% dim
```

```
## [1] 1000    5
```

```
gn_seq$folds %>% length
```

```
## [1] 5
```

Then we could use `ctmleGeneral` algorithm. As input estimator is already trained, it is much faster than previous C-TMLE algorithms.

*Note: we recommend use the same `folds` as `build_gn_seq` for `ctmleGeneral`, to make cross-validation objective.*

```
ctmle_general_fit1 <- ctmleGeneral(Y = Y, A = A, W = W, Q = Q,
                                ctmletype = 1,
                                gn_candidates = gn_seq$gn_candidates,
                                gn_candidates_cv = gn_seq$gn_candidates_cv,
                                folds = folds, v = 5)
```

```
ctmle_general_fit1
```

```
## C-TMLE result:
## parameter estimate: 2.19494
## estimated variance: 0.08348
## p-value: 3.0302e-14
## 95% conf interval: (1.62865, 2.76122)
```

# Citation

If you used `ctmle` package in your research, please cite:

Ju, Cheng; Susan, Gruber; van der Laan, Mark J.; ctmle: Variable and Model Selection for Causal Inference with Collaborative Targeted Maximum Likelihood Estimation

## References

### C-TMLE LASSO and C-TMLE for Model Selection

Ju, Cheng; Benkeser, David; van der Laan, Mark; “Robust inference on the average treatment effect using the outcome highly adaptive lasso”, Biometrics, <https://doi.org/10.1111/biom.13121>

### Scalable Discrete C-TMLE with Pre-ordering

Ju, Cheng; Gruber, Susan; Lendle, S. D.; et al. Scalable collaborative targeted learning for high-dimensional data. Statistical methods in medical research, 2019, 28(2): 532-554.

### Discrete C-TMLE with Greedy Search

Susan, Gruber, and van der Laan, Mark J.. “An Application of Collaborative Targeted Maximum Likelihood Estimation in Causal Inference and Genomics.” The International Journal of Biostatistics 6.1 (2010): 1-31.

### General Template of C-TMLE

van der Laan, Mark J., and Susan Gruber. “Collaborative double robust targeted maximum likelihood estimation.” The international journal of biostatistics 6.1 (2010): 1-71.